

Streaming Framework

(Part 2)

OUTLINE

- ① Introduction to the streaming model (Part 1)
- ② Warm-up: finding the majority element (Part 1)
- ③ Sampling (Part 1)
 - Uniform sampling with reservoir sampling
 - Finding (approximate) frequent items
- ④ Sketching
 - Counting distinct elements with probabilistic counting
 - Estimating individual frequencies and the second moment
- ⑤ Filtering: Bloom filters for membership problem

Sketching

Sketch: space-efficient data structure that can be used to provide (typically **probabilistic**) estimates of (statistical) characteristics of a data stream.

Frequency Moments

Consider a **stream** $\Sigma = x_1, x_2, \dots, x_n$, whose elements belong to a **universe** U . (In fact, Σ could represent an arbitrary prefix of a longer, possibly unbounded, stream.)

For each $u \in U$ occurring in Σ define its (*absolute*) *frequency*

$$f_u = |\{j : x_j = u, 1 \leq j \leq n\}|,$$

i.e., the number of occurrences of u in Σ

Definition: Frequency Moments

For every $k \geq 0$, the k -th frequency moment F_k of Σ is

$$F_k = \sum_{u \in U} f_u^k.$$

(We assume $0^0 = 0$)

Frequency Moments

Relevant informations from frequency moments:

- $F_0 = \text{number of distinct items in } \Sigma$. It is useful, for instance, in the analysis of web-logs.
- $F_1 = |\Sigma|$: trivial to maintain with a counter.
- $1 - F_2/|\Sigma|^2 = \text{Gini index of } \Sigma$. It provides info on data impurity: the closer to 1 and the higher is the impurity (i.e., diversity). It is used for decision tree induction.

Observations on the GINI INDEX

* If $x_1 = x_2 = \dots = x_n$ (MIN IMPURITY) then

$$F_2 = n^2 = |\Sigma|^2 \Rightarrow \text{GINI INDEX} = 1 - F_2/|\Sigma|^2 = 0$$

* If Σ contains m distinct elements each occurring n/m times (HIGH IMPURITY for large m) then

$$F_2 = m \cdot (n/m)^2 = n^2/m \Rightarrow$$

$$\text{GINI INDEX} = 1 - F_2/|\Sigma|^2 = 1 - (n^2/m)/n^2 = 1 - 1/m$$

which tends to 1 as m grows large

Estimating F_0 for Σ (i.e., distinct elements)

Exact computation: use $|U|$ counters or dictionary with $|F_0|$ elements
(not suited for streaming setting).

Probabilistic Counting Algorithm ([Flajolet, Martin 1983])

- Working memory $O(\log |U|)$.
- F_0 approximated within a factor c , with probability $\geq 1 - 2/c$
(accuracy-confidence tradeoff).
- **Main idea:**
 - Map each $u \in U$ to a hash value $h(u) \in [0, |U| - 1]$.
 $\Rightarrow h(u)$ is a $O(\log |U|)$ -bit binary string.
 - Keep track of max number of trailing 0's in a hash code, which grows, probabilistically, with the number of distinct elements.

Probabilistic Counting Algorithm

Ingredients:

- Integer $R \in [0, \lceil \log_2 |U| \rceil]$, which requires $\Theta(\log \log |U|)$ bits.
- Hash Function $h : U \rightarrow [0..|U| - 1]$. We assume:
 - For every $u \in U$, $h(u)$ has uniform distribution in $[0..|U| - 1]$
 - All $h(u)$'s are pairwise independent.
- Notation: for $i \in [0..|U| - 1]$, define

$tr(i)$ = number of trailing zeroes in binary representation of i

e.g., $12 = (1100)_2 \Rightarrow tr(12) = 2$

Initialization : $R \leftarrow 0$

Algorithm: For each $x_t \in \Sigma$ do $R \leftarrow \max\{R, tr(h(x_t))\}$

After processing x_n , estimate F_0 as

$$\tilde{F}_0 = 2^R.$$

Example: $\Sigma = A, D, A, A, C, B, F, F, B, A, E, C$

$U = \text{Alphabet} \Rightarrow |U| = 26 \quad \lceil \log_2 |U| \rceil = 5$

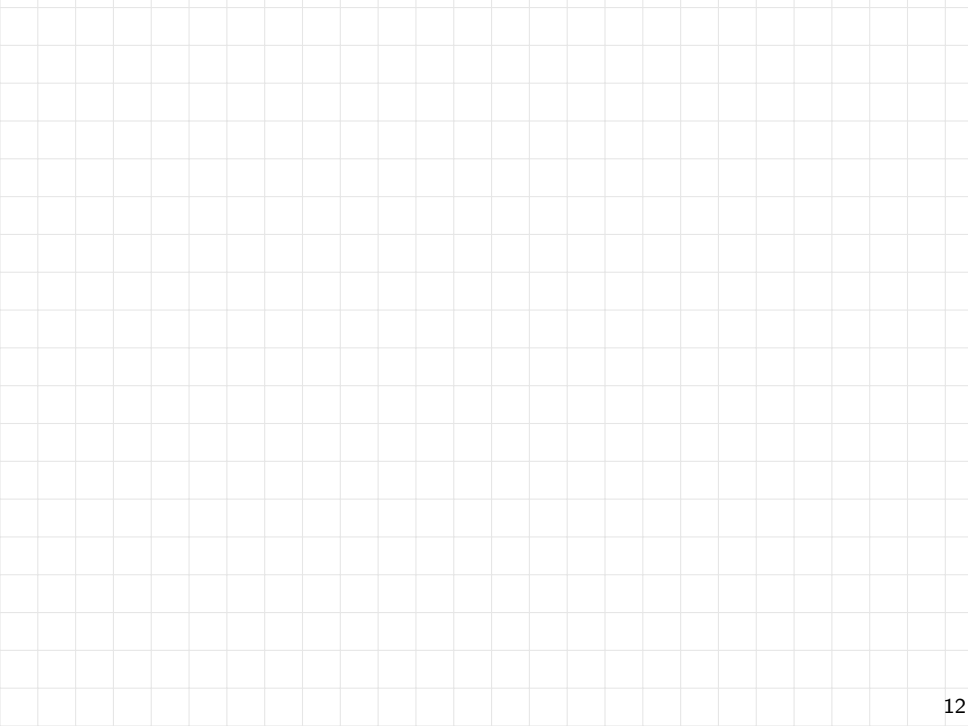
$\Rightarrow R \in [0, 5]$

x	$h(x)$	$t_2(h(x))$	R
A	01001	0	0
D	11100	2	2
C	11000	3	3
B	01110	1	3
F	10100	2	3
E	10011	0	3

After processing the entire stream Σ :

Estimate $\tilde{F}_0 = 2^R = 2^3 = 8$

True Value $F_0 = 6$



Why does it work? (Intuition)

For simplicity, assume $|U|$ a power of 2.

For $x \in \Sigma$, what is the probability that $h(x)$ has at least j trailing zeros?

- $\text{Prob}(tr(h(x)) \geq 1) = 1/2$

- $\text{Prob}(tr(h(x)) \geq 2) = 1/4$

.....

- $\text{Prob}(tr(h(x)) \geq j) = 1/2^j$

The fact that for any $x \in \Sigma$ $P_2(t_2(h(x)) \geq j) = 1/2^j$ is easily proved by regarding $h(x)$ as a string of $\lceil \log_2 |U| \rceil$ random and independent bits: thus $t_2(h(x)) \geq j$ iff the first j bits are 0, irrespective of the other bits, which happens with probability $1/2^j$.

This has 2 consequences

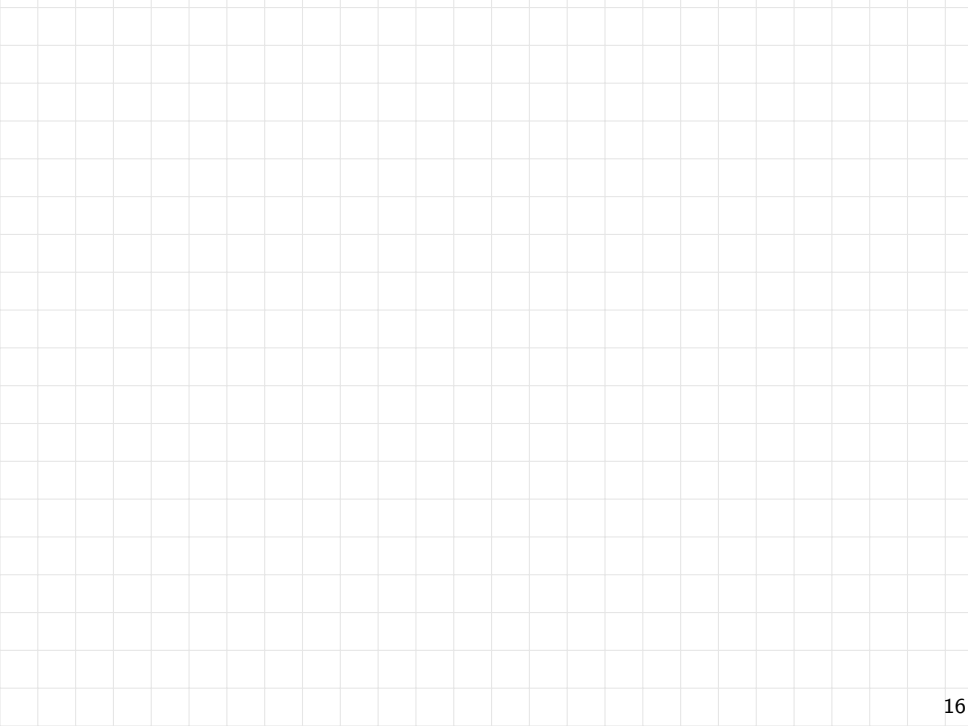
$$\begin{aligned}
 \text{(A)} \quad & P_2(\text{At the end of } \Sigma, R \geq j) = \\
 & = P_2(\exists \text{ an item } x \in \Sigma: t_2(h(x)) \geq j) \\
 & \leq F_0 \cdot 1/2^j \text{ (union bound on the distinct elements in } \Sigma)
 \end{aligned}$$

(B) Among the F_0 random hash values associated with the distinct elements of Σ , those whose binary configurations have $\geq j$ trailing 0's is $F_0 \cdot (1/2^j)$ in EXPECTATION

$\Rightarrow$ It is reasonable to expect that R is such that

$$\frac{F_0}{2^R} \geq 1 \quad \text{and for } R' > R \quad \frac{F_0}{2^{R'}} < 1$$

$\Rightarrow 2^R \approx F_0$ which provides an intuitive justification for the algorithm



Probabilistic guarantees

The following theorem summarizes the properties of the algorithm.

Theorem

For a stream Σ of n elements, the **probabilistic counting algorithm** returns a value $\tilde{F}_0$ such that, for any $c > 2$:

$$\Pr(\tilde{F}_0 < F_0/c) \leq 1/c \quad \text{and} \quad \Pr(\tilde{F}_0 > cF_0) \leq 1/c,$$

hence

$$\Pr(F_0/c \leq \tilde{F}_0 \leq cF_0) \geq 1 - 2/c.$$

The algorithm requires a working memory of $O(\log |U|)$ bits, 1 pass, and $O(\log |U|)$ update/query time per element.

Proof:

$$* P_2(\tilde{F}_0 > c F_0) \leq 1/c$$

$$P_2(\tilde{F}_0 > c F_0) = P_2(2^R > c F_0)$$

$$= P_2(R > \log_2(c F_0))$$

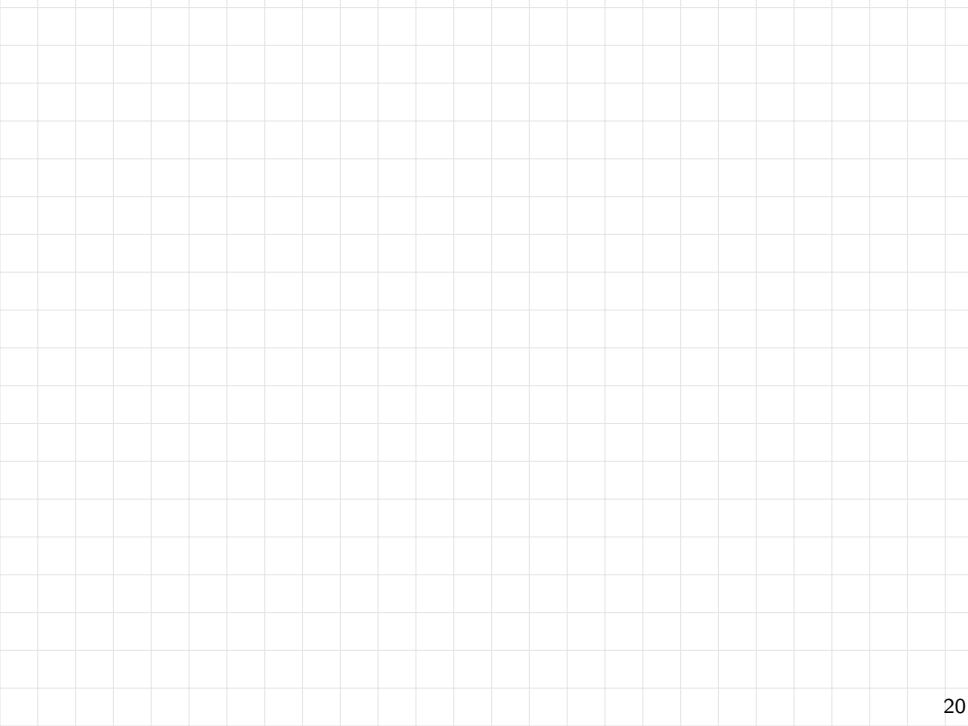
$$\leq F_0(1/2^{\log_2 c F_0}) = \frac{1}{c} \quad (\text{see (A) from slide 14})$$

$$* P_2(\tilde{F}_0 < F_0/c) \leq 1/c \quad \text{we skip this proof}$$

- * Working memory: $O(\log |U|)$
 - R requires $O(\log \log |U|)$ bits
 - $h(x) \in [0, |U|-1] \Rightarrow$ requires $O(\log |U|)$ bits

* Update time: $O(\log |U|)$ to process the binary configuration of $h(x_t)$

* Query time: $O(\log |U|)$ to compute 2^R
B



Exercises

Exercise: High Probability Guarantees (median trick)

Consider a stream Σ with elements from a universe U and let F_0 be the number of distinct elements in Σ . Suppose that you run ℓ *independent instances* of the probabilistic counting algorithm, obtaining ℓ estimates $\tilde{F}_0^{(j)}$ for F_0 , with $1 \leq j \leq \ell$. Assume ℓ odd and let $\tilde{F}_0$ be the median value of the $\tilde{F}_0^{(j)}$'s. Show that with a suitable choice of $\ell = \Theta(\log |U|)$ we have:

$$\Pr(\tilde{F}_0 < F_0/16) \leq 1/|U| \quad \text{and} \quad \Pr(\tilde{F}_0 > 16F_0) \leq 1/|U|.$$

Hint: Use the previous theorem and the Chernoff bound.

Estimating individual frequencies and F_2 for Σ

OBJECTIVE: in one pass over Σ , we want to compute a **small sketch** that enables to derive **unbiased probabilistic estimates** of

- f_u for any given $u \in U$ (*individual frequencies*)
- $F_2 = \sum_{u \in U} (f_u)^2$ (*second moment*)

with **provable space-accuracy tradeoffs**

Observation: clearly, the **exact computation** of all f_u 's or of F_2 might require space proportional to $|\Sigma|$.

Count-min sketch

The first approach we consider is based on the **count-min sketch** invented by [Cormode,Muthukrishnan 2003].

Main ingredients

- $d \times w$ array C of counters ($O(\log n)$ bits each)
- d hash functions: $h_0, h_1, \dots, h_{d-1}$, with

$$h_j : U \rightarrow \{0, 1, \dots, w - 1\},$$

for every j .

Count-min sketch: algorithm

Initialization: $C[j, k] = 0$, for every $0 \leq j < d$ and $0 \leq k < w$.

For each x_t in Σ do

For $0 \leq j \leq d - 1$ do $C[j, h_j(x_t)] \leftarrow C[j, h_j(x_t)] + 1$;

At the end of the stream: for any $u \in U$, its frequency f_u can be estimated as:

$$\tilde{f}_u = \min_{0 \leq j \leq d-1} C[j, h_j(u)].$$

Observations:

- d and w are *design parameters* that regulate the tradeoff between space, time, and accuracy.
- By construction, for any $u \in U$ and $0 \leq j < d$, we clearly have $C[j, h_j(u)] \geq f_u$. This justifies the minimum, but also shows that

$\tilde{f}_u$ is a **BIASED ESTIMATOR!**

Example: $n = 15, d = 3, w = 3$

$\Sigma = A, B, C, B, D, A, C, D, A, B, D, C, A, A, B$

u, f_u	h_0	h_1	h_2
A, 5	0	1	1
B, 4	1	2	1
C, 3	0	0	2
D, 3	1	1	2

Array C		
8 = $5A + 3C$	7 = $4B + 3D$	0
3 = $3C$	8 = $6A + 3D$	4 = $4B$
0	9 = $5A + 4B$	6 = $3C + 3D$

row 0

row 1

row 2

- $\tilde{f}_A = \min\{8, 8, 9\} = 8 > f_A = 5$ $h_0(A) = 0$
- $\tilde{f}_B = \min\{7, 4, 9\} = 4 = f_B$
- $\tilde{f}_C = \min\{8, 3, 6\} = 3 = f_C$
- $\tilde{f}_D = \min\{7, 8, 6\} = 6 > f_D = 3$

Count-min sketch: analysis of accuracy

We assume that:

- For each $j \in [0, d - 1]$ and each $u, v \in U$ with $u \neq v$, $h_j(u)$ and $h_j(v)$ are independent random variables uniformly distributed in $[0, w - 1]$.
- The d hash functions $h_1, h_2, \dots, h_d$ are mutually independent.

Theorem

Consider a $d \times w$ count-min sketch for a stream Σ of length n , where $d = \log_2(1/\delta)$ and $w = 2/\epsilon$, for some $\delta, \epsilon \in (0, 1)$. The sketch ensures that for any given $u \in U$ occurring in Σ

$$\tilde{f}_u - f_u \leq \epsilon \cdot n,$$

with probability $\geq 1 - \delta$.

Obs.: The bias in the estimated frequencies discourages their use to estimate the second moment F_2 .

APPLICATIONS of the COUNT-MIN SKETCH

1) **HEAVY HITTERS**: the sketch allows to quickly and effectively identify frequent items ($f_u \geq \varphi n$) detecting each such item as soon as $\bar{f}_u \geq \varphi n$ rather than a posteriori which would be expensive.

2) **QUERY PLANNING**. In a relational database query that requires joining several large tables, the order of execution of the joins may heavily effect the running time. To choose the best ordering it is important to estimate

the resulting size of each individual join. For example, consider the join of 2 tables A, B with a common attribute x

SQL query:
SELECT *
FROM A, B
WHERE A.x = B.x

To estimate the number of rows in the resulting table do:

* Create a dxm Count-min Sketch C_A for A.x

* Create a dxm Count-min Sketch C_B for B.x

IMPORTANT: use the same d hash functions for C_A and C_B

An estimate of the number of rows in the table resulting from the join is given by:

$$\min_{0 \leq j < d} \sum_{k=0}^{w-1} C_A[j, k] * C_B[j, k]$$

PROOF of THEOREM : let's fix an arbitrary item $u \in \Sigma$
for each row j ($0 \leq j < d$) the error in the estimate of f_u
provided by row j is $C[j, h_j(u)] - f_u$. Note that
 $C[j, h_j(u)]$ receives "in expectation" a fraction w

of the entire mass n ($\equiv$ total number of elements in Σ)

Precisely

$$E[C[j, h_j(w)] - f_u] = \sum_{\substack{a \in U \\ a \neq u}} f_a \cdot P_2(h_j(a) = h_j(u))$$

$$= \sum_{\substack{a \in U \\ a \neq u}} f_a \frac{1}{w} \leq \sum_{a \in U} f_a \frac{1}{w} = \frac{n}{w} = \frac{n \cdot \varepsilon}{2}$$

Now we are interested in $P_2(C[j, h_j(w)] - f_u > \varepsilon n)$

$$P_2(C[\bar{L}_j, h_j(u)] - f_u > \varepsilon n) = P_2(C[\bar{L}_j, h_j(u)] - f_u > \frac{2\varepsilon n}{2})$$

$$\leq P_2(C[\bar{L}_j, h_j(u)] - f_u > 2E[C[\bar{L}_j, h_j(u)] - f_u])$$

since $E[C[\bar{L}_j, h_j(u)] - f_u] \leq \varepsilon n/2$

$$\leq P_2(C[\bar{L}_j, h_j(u)] - f_u \geq 2E[C[\bar{L}_j, h_j(u)] - f_u])$$

$$\leq \frac{1}{2} \quad \text{by Markov's inequality}$$

Final steps:

$$P_2(\tilde{f}_u - f_u > \varepsilon n) = P_2(\forall 0 \leq j < d \quad C[\bar{L}_j, h_j(u)] - f_u > \varepsilon n)$$

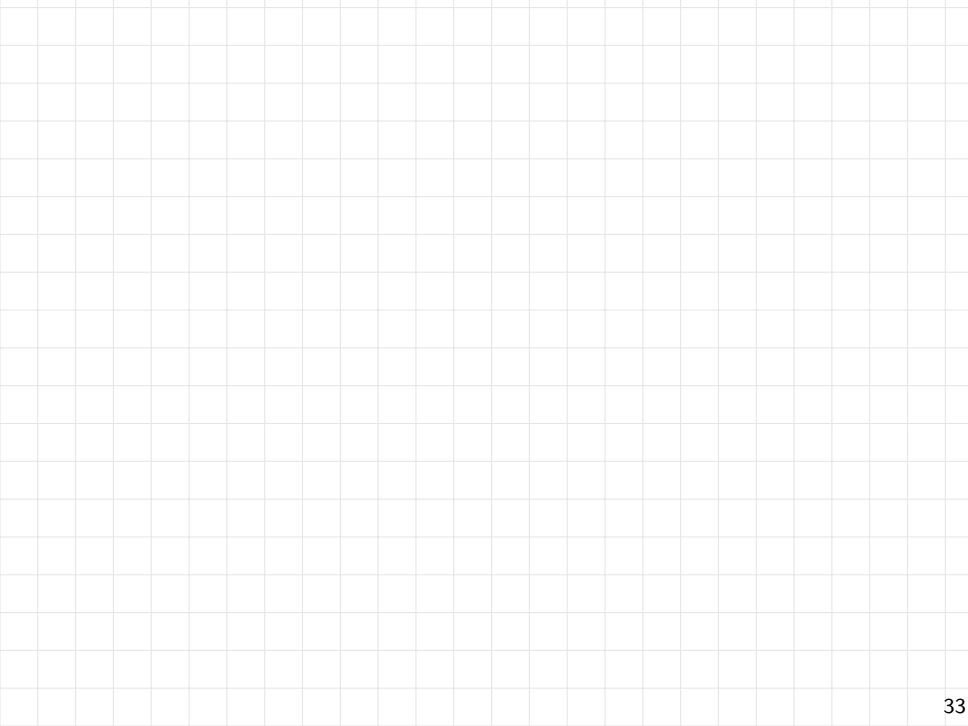
$$= \prod_{j=0}^{d-1} P_2(C[j, h_j(u)] - f_u > \varepsilon n) \quad \text{by independence of the } h_j \text{'s}$$

$$\leq \left(\frac{1}{2}\right)^d$$

$$\text{Since } d = \log_2(1/\delta) \quad \left(\frac{1}{2}\right)^d = \delta$$

$$\text{Therefore, } P_2(\tilde{f}_u - f_u \leq \varepsilon n) \geq 1 - \delta$$

□



Count sketch

The **count sketch** was invented by [Charikar,Chen,Farach-Colton 2002], and can be seen as an **unbiased variant of the count-min sketch**.

IDEA: for each item $u \in U$ multiply its contributions to each row by a value in $\{-1, +1\}$ randomly selected, so to **cancel out collisions**.

Main ingredients

- $d \times w$ array C of counters ($O(\log n)$ bits each)
- d hash functions: $h_0, h_1, \dots, h_{d-1}$, with

$$h_j : U \rightarrow \{0, 1, \dots, w-1\},$$

for every j .

- d hash functions: $g_0, g_1, \dots, g_{d-1}$, with

$$g_j : U \rightarrow \{-1, +1\},$$

for every j .

Count sketch: algorithm

Initialization: $C[j, k] = 0$, for every $0 \leq j < d$ and $0 \leq k < w$.

For each x_t **in** Σ **do**

For $0 \leq j \leq d - 1$ **do** $C[j, h_j(x_t)] \leftarrow C[j, h_j(x_t)] + g_j(x_t)$;

At the end of the stream: for any $u \in U$ and $0 \leq j < d$, let

$$\tilde{f}_{u,j} = g_j(u) \cdot C[j, h_j(u)].$$

The frequency of u can be estimated as:

$$\tilde{f}_u = \text{median of the } \tilde{f}_{u,j} \text{'s}$$

Example: $n = 15, d = 3, w = 3$

$\Sigma = A, B, C, B, D, A, C, D, A, B, D, C, A, A, B$

u, f_u	h_0	g_0	h_1	g_1	h_2	g_2
A, 5	0	+1	1	+1	1	+1
B, 4	1	-1	2	+1	1	-1
C, 3	0	-1	0	-1	2	+1
D, 3	1	-1	1	+1	2	+1

Array C		
row 0	2 = $5A - 3C$	-7 = $-4B - 3D$
row 1	-3 = $-3C$	8 = $5A + 3D$
row 2	0	1 = $5A - 4B$
		6 = $3C + 3D$

- $\tilde{f}_A = \text{median} \{2, 8, 1\} = 2 < f_A = 5$

- $\tilde{f}_B = \text{median} \{7, 4, -1\} = 4 = f_B$

- $\tilde{f}_C = \text{median} \{-2, 3, 6\} = 3 = f_C$

- $\tilde{f}_D = \text{median} \{7, 8, 6\} = 7 > f_D = 3$

Count sketch: analysis of accuracy

Assumptions: for both sets of hash functions (the h_j 's and the g_j 's) we make the same assumptions of **independence and uniform distribution**, which we made for the h_j 's in the analysis of the count-min sketch.

Theorem

Consider a $d \times w$ count sketch for a stream Σ of length n , where $d = \log_2(1/\delta)$ and $w = O(1/\epsilon^2)$, for some $\delta, \epsilon \in (0, 1)$. The sketch ensures that for any given $u \in U$ occurring in Σ :

- 1 $E[\tilde{f}_{u,j}] = f_u$, for any $j \in [0, d-1]$, i.e., $\tilde{f}_{u,j}$ is an unbiased estimator of f_u ;
- 2 With probability $\geq 1 - \delta$,

$$|\tilde{f}_u - f_u| \leq \epsilon \cdot \sqrt{F_2},$$

where $F_2 = \sum_{u \in U} (f_u)^2$ (true second moment).

} we skip the proof of this part

Intuition. Due to the random signs, on average the “noise” created by several items colliding on the same column as a u , cancel out.

COMPARISON: Count-min VS Count Sketch

BIAS $\downarrow$ biased $\downarrow$ unbiased

ACCURACY For any $u \in U$, with prob $\geq 1 - \delta$

Count-min: $\tilde{f}_u - f_u \leq \epsilon n$

Count sketch: $|\tilde{f}_u - f_u| \leq \epsilon \sqrt{F_2}$

$$\text{Now: } F_2 = \sum_{u \in U} (f_u)^2 \leq \left(\sum_{u \in U} f_u \right)^2 = n^2 \Rightarrow \sqrt{F_2} \leq n$$

So, Count Sketch has an error bound not worse than the one of Count-min (but uses a larger w)

If the distribution of the frequencies is not too skewed, however, we can have $\sqrt{F_2} \ll n$ and the error bound for Count sketch becomes much lower

PROOF of THEOREM (only Point ①)

Fix $u \in U$ and $j \in [0, d-1]$ arbitrarily. We now show that $E[\tilde{f}_{u,j}] = f_u$. Recall that $\tilde{f}_{u,j} = g_j(u) \cdot C[j, h_j(u)]$

For every $a \in U$ $a \neq u$ we define a random variable Y_a which represents the contribution of a to $\tilde{f}_{u,j}$

$$Y_a = \begin{cases} f_a & \text{if } h_j(a) = h_j(u) \text{ AND } g_j(a) = g_j(u) \\ -f_a & \text{if } h_j(a) = h_j(u) \text{ AND } g_j(a) = -g_j(u) \\ 0 & \text{otherwise} \end{cases}$$

It is easy to see that $\tilde{f}_{u,j} = f_u + \sum_{\substack{a \in U \\ a \neq u}} Y_a$

Now observe that

$$\left. \begin{aligned} * P_2(Y_a = f_a) &= \frac{1}{w} \cdot \frac{1}{2} \\ * P_2(Y_a = -f_a) &= \frac{1}{w} \cdot \frac{1}{2} \end{aligned} \right\} \text{their sum is } 1/w$$

$$* P_2(Y_a = 0) = 1 - 1/w$$

We have :

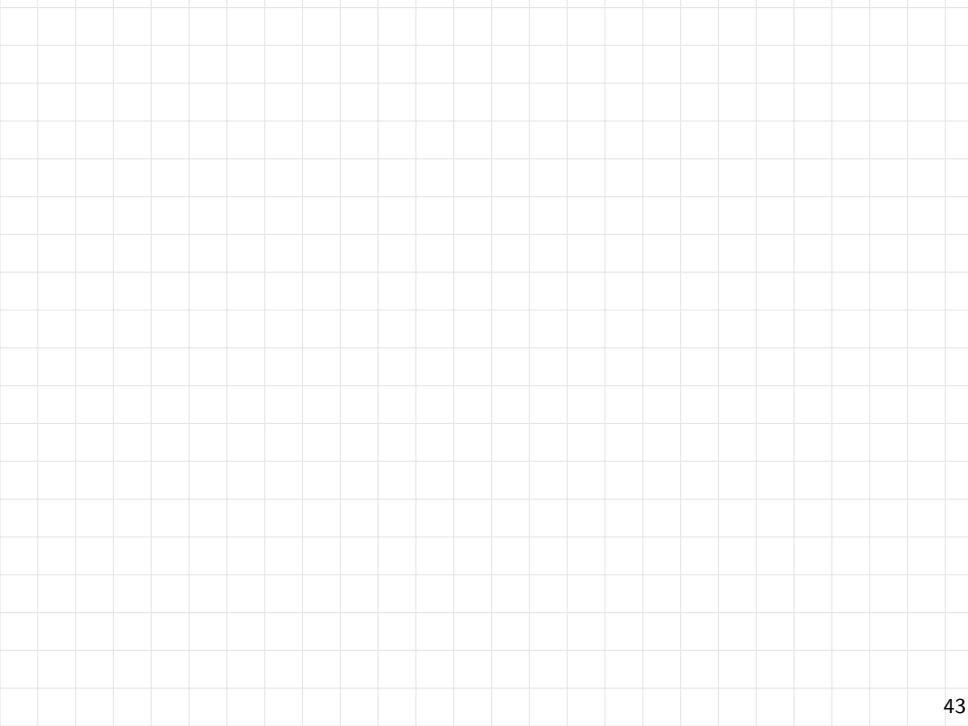
$$E[\tilde{f}_{u,j}] = E[f_u + \sum_{\substack{a \in U \\ a \neq u}} Y_a] = f_u + E[\sum_{\substack{a \in U \\ a \neq u}} f_a]$$

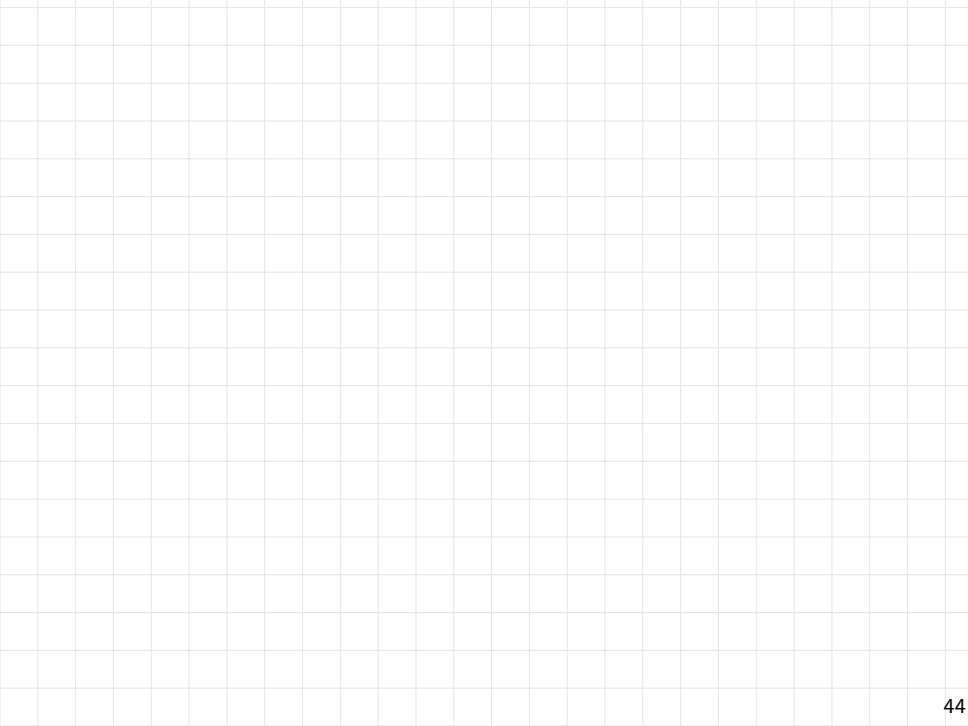
$$= f_u + \sum_{\substack{a \in U \\ a \neq u}} E[Y_a] \quad \text{by linearity of expectation}$$

$$\text{And } E[Y_a] = f_a \cdot \frac{1}{w} \cdot \frac{1}{2} - f_a \cdot \frac{1}{w} \cdot \frac{1}{2} + 0 \cdot (1 - 1/w) = 0$$

$$\Rightarrow E[\tilde{f}_{u,j}] = f_u + \sum_{\substack{a \in U \\ a \neq u}} 0 = f_u$$

□





Estimation of F_2

Given a $d \times w$ count sketch for Σ , define

$$\tilde{F}_{2,j} = \sum_{k=0}^{w-1} (C[j, k])^2 \quad \text{for } 0 \leq j < d$$

We can derive the following estimate for the true second moment F_2 :

$$\tilde{F}_2 = \text{median of the } \tilde{F}_{2,j} \text{'s}$$

Example (same as before)

$\Sigma = A, B, C, B, D, A, C, D, A, B, D, C, A, A, B$

$$F_2 = (f_A)^2 + (f_B)^2 + (f_C)^2 + (f_D)^2 = 5^2 + 4^2 + 3^2 + 3^2 = 59.$$

Array C from before		
2	-7	0
-3	8	4
0	1	6

- Estimate from row $j = 0$: $2^2 + (-7)^2 = 53$
- Estimate from row $j = 1$: $(-3)^2 + 8^2 + 4^2 = 89$
- Estimate from row $j = 2$: $0^2 + 1^2 + 6^2 = 37$

$$\Rightarrow \tilde{F}_2 = 53 \quad (\text{True value} = 59)$$

Accuracy of $\tilde{F}_2$

The following theorem can be proved under the same assumptions made for the analysis of the count sketch

Theorem

Consider a $d \times w$ count sketch for a stream Σ of length n , where $d = \log_2(1/\delta)$ and $w = O(1/\epsilon^2)$, for some $\delta, \epsilon \in (0, 1)$. The sketch ensures that:

- 1 $E[\tilde{F}_{2,j}] = F_2$ for any $0 \leq j < d$. That is, any $\tilde{F}_{2,j}$ is an unbiased estimator of F_2 . **EXERCISE**

- 2 With probability $\geq 1 - \delta$,

$$|\tilde{F}_2 - F_2| \leq \epsilon \cdot \sqrt{F_2}.$$

} we skip the proof of this point

Proof of Point ① (only a hint)

Follow these steps. Fix an arbitrary row j

1) $\forall a, b \in U$ $a \neq b$ define

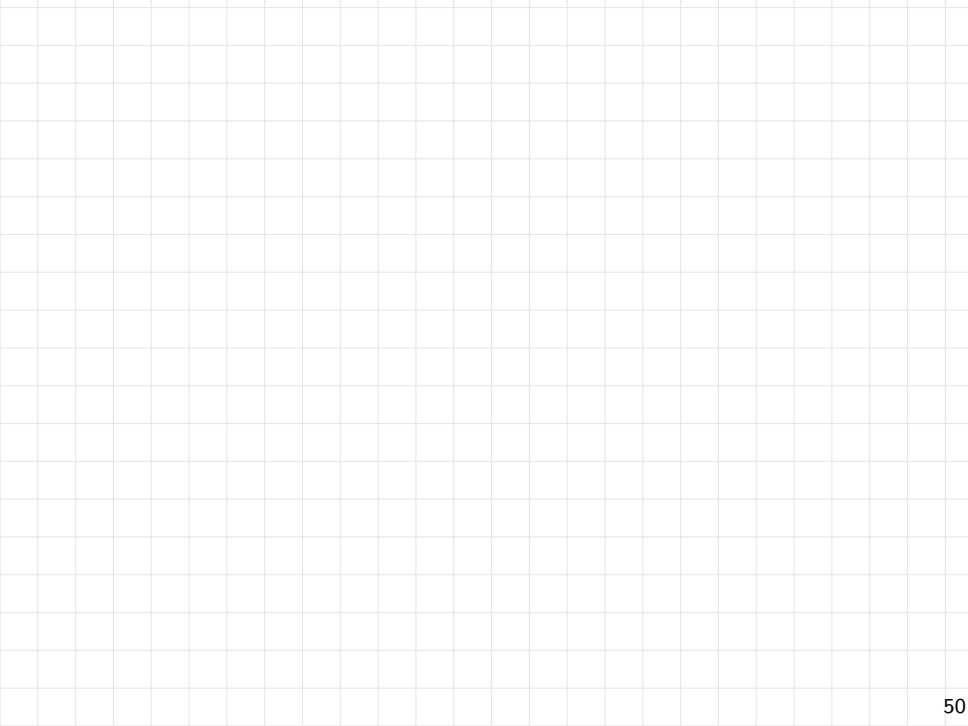
$$Y_{ab} = \begin{cases} f_a f_b & \text{if } h_j(a) = h_j(b) \text{ AND } g_j(a) = g_j(b) \\ -f_a f_b & \text{if } h_j(a) = h_j(b) \text{ AND } g_j(a) = -g_j(b) \\ 0 & \text{otherwise} \end{cases}$$

So, Y_{ab} relates to the noise contributed by a and b to $\tilde{F}_{2,j}$

2) Argue that $\tilde{F}_{2j} = \sum_{a \in U} f_a^2 + \sum_{\substack{a, b \in U \\ a \neq b}} 2Y_{ab}$

(3) Show that $E[Y_{ab}] = 0 \quad \forall a, b \in U \quad a \neq b$

(4) Use (2) and (3) to conclude the proof



Analysis of performance metrics

Both **count-min** and **count sketches** can be computed in **1 pass**

To assess space and time performance, we assume:

- Each hash function can be applied in constant time
- The space occupied by the sketch dominates over the one needed to store the hash functions

For both sketches we have:

- **Working memory:** $O(d \cdot w)$, which becomes $O(\log(1/\delta)/\epsilon)$, for the count-min sketch, and $O(\log(1/\delta)/\epsilon^2)$, for the count sketch, in order to attain the probabilistic accuracy stated before.
- **Update time:** $O(d) = O(\log(1/\delta))$,
- **Query time:** any $\tilde{f}_u$ (individual frequency) is computed in $O(d)$ time, while $\tilde{F}_2$ (second moment) in $O(d \cdot w)$ time.

Filtering

Motivation

For many applications, processing a data stream $\Sigma = x_1, x_2, \dots$ entails essentially the identification of the x_i 's which meet a certain criterion.

Some criteria can be checked very easily with a minimum cost in terms of space and time. However, this is not always the case.

Example. Suppose that the x_i 's are email addresses and that when x_i arrives we need to check whether it belongs to a set S of verified addresses. If S is very large (e.g., 1 billion addresses of approximately 20 bytes each), we face two issues:

- If S does not fit into main memory, it must be stored on disk.
- Standard exact techniques to check $x_i \in S$, especially if S is on disk, may be time consuming and not compatible with a high arrival rate.

Can we check membership efficiently with reasonable accuracy?

Bloom filter

Approximate membership problem

Given a stream $\Sigma = x_1, x_2, \dots$ of elements from some universe U , and let S be a set of m elements from U . Store S into a compact data structure that, for any given x_t , allows to check whether $x_t \in S$ with

- no error, when $x_t \in S$ (No false negatives)
- small probability error, when $x_t \notin S$ (Small false positive rate)

A solution to the problem comes from the **Bloom filter**, introduced in [Bloom 1970]. Its **main ingredients** are:

- Array A of n bits, all initially 0.
- k hash functions: $h_0, h_1, \dots, h_{k-1}$, with

$$h_j : U \rightarrow \{0, 1, \dots, n-1\} \quad \text{for every } 0 \leq j < k$$

Note that n and k are *design parameters* that regulate the tradeoff between space/time and accuracy.

Bloom filter

Initialization of the bit array A :

for each $e \in S$ do

for $0 \leq j < k$ do $A[h_j(e)] \leftarrow 1$;

Membership test: for any x_t in Σ

if $A[h_0(x_t)] = A[h_1(x_t)] = \dots = A[h_{k-1}(x_t)] = 1$ then claim $x_t \in S$
else claim $x_t \notin S$

Straightforward properties:

- The approach ensures that there are **no false negatives**
- Assuming that $k \ll n$, and that the hash functions can be stored compactly, the **required working memory** is dominated by the storage of $A \Rightarrow n$ bits.
- Assuming that each hash function can be applied in $O(1)$ time, the **membership test requires $O(k)$ time**.

Example

- $S = \{X, Y, Z\}$
- $n = 12$
- $k = 2$

	X	Y	Z
h_0	0	1	4
h_1	4	9	9

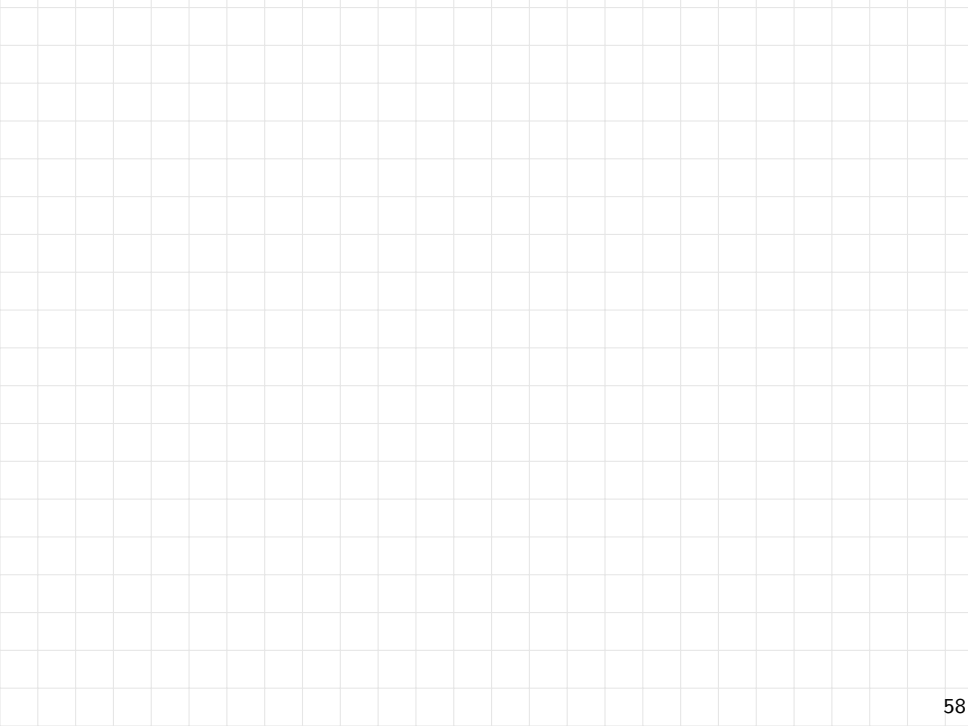
The resulting array is:

$A \equiv$

0	1	2	3	4	5	6	7	8	9	10	11
1	1	0	0	1	0	0	0	0	1	0	0

Blue arrows point from the array to the sets: X at index 0, Y at index 1, X, Z at index 4, and Y, Z at index 9.

Check (X)	h_0 0	h_1 4	Result YES	True positive
Check (T)	4	7	NO	True negative
Check (P)	1	9	YES	False positive



Bloom filter: analysis of false positive rate

Assumptions: for the set of hash functions (the h_j 's) we make the same assumptions of **independence and uniform distribution**, which we made in the analysis of the count-min sketch.

Theorem

Suppose that n is sufficiently large. For any given x_i which does not belong to S , the probability that x_i is erroneously claimed to be in S is

$$\Pr(A[h_j(x_i)] = 1 \text{ for each } 0 \leq j < k) \simeq (1 - e^{-km/n})^k$$

This probability is referred to as **false positive rate**.

Email example: In the case of email addresses mentioned before, $m = 10^9$ and storing the entire set S would require **20GB** (assuming that each email takes 20 bytes). Using a Bloom filter with $n = 8m$ (hence $|A| = 1\text{GB}$), and $k = 6$, the false positive rate is about 2.15%.

address

Proof Assuming that the h_{ij} 's are independent and uniform in $[0, n-1]$, after the initialization of A the indices of A which are set to 1 can be seen as $k \cdot m$ independent random variables in $[0, n-1]$. With this observation we can now compute the probability that $A[l] = 0$, for any $l \in [0, n-1]$ (after the initialization)

$$P(A[u]=0) = \left(1 - \frac{1}{n}\right)^{km}$$

$$= \left(1 - \frac{1}{n}\right)^{n \cdot \frac{km}{n}}$$

$$= \left(\left(1 - \frac{1}{n}\right)^n\right)^{\frac{km}{n}} \approx e^{-km/n}$$

for large n

Define $p = e^{-km/n}$. We make the SIMPLIFYING

ASSUMPTION that A contains EXACTLY $p \cdot n$ 0's

(The theorem can be proved also without the assumption)

Consider an element $x \in \Sigma$ such that $x \notin S$

let $l_j = h_j(x)$ $0 \leq j < k$

Since l_j is uniformly distributed in $[0, n-1]$
we have that

$$P_2(A[l_j] = 1) = 1 - \frac{pn}{n} = 1 - p$$

← Probab. lity of not hitting a 0

Then for

$$\begin{aligned} P_2(A[l_0] = A[l_1] = \dots = A[l_k] = 1) &= (1-p)^k \\ &= \left(1 - e^{-kn/n}\right)^k \end{aligned}$$

□

OBSERVATIONS It can be shown that for fixed n, m , the FALSE POSITIVE RATE (FPR) is minimized for $k = (\ln 2) n/m$. With this value of k , the FPR becomes

$$(1 - e^{-km/n})^k = \left(\frac{1}{2}\right)^k = (0.6185)^{n/m}$$

For example

$n = 8m$ and $k = 5 \approx 6$ we get

FPR $\in [0.01, 0.03]$ between 1% - 3%

An alternative approach to approximate membership is through FINGERPRINTING

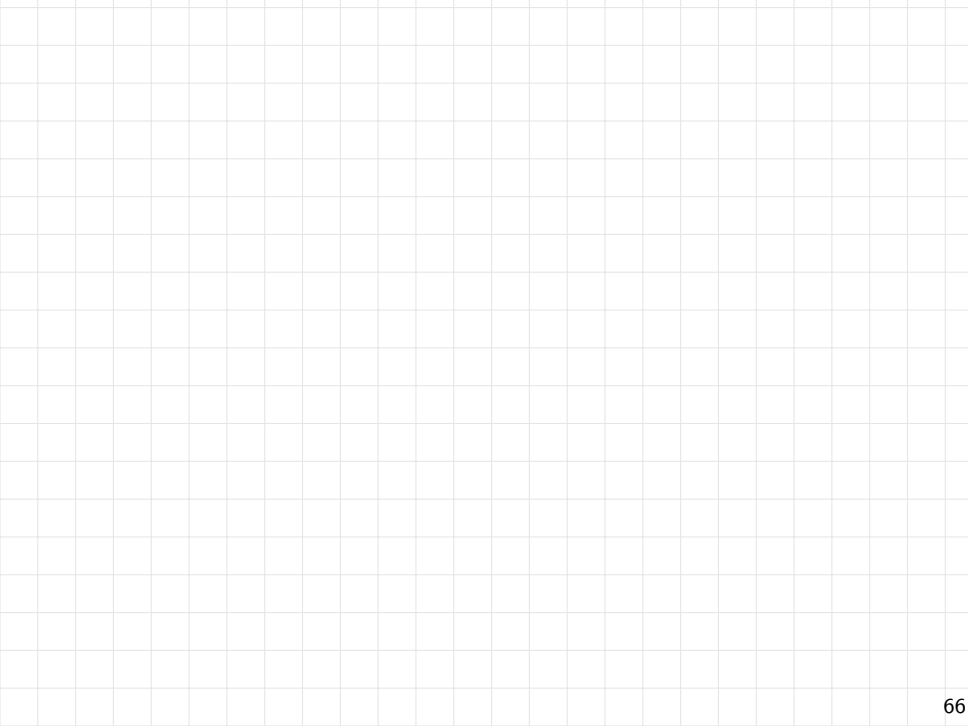
- * Rep each $x \in S$ into an integer $[0, 2^b - 1]$

- * let $r_1, r_2, \dots, r_m$ be integers (FINGERPRINTS) associated to the elements of S

- * A new element $x \in \Sigma$ is deemed to be in S iff its fingerprint is equal to some r_i

To store the r_i 's we need $\Theta(m \cdot b)$ bits

It can be shown that to get $FPR = 0.02$
 b must be $\Omega(\log m)$ while
with the Bloom filter using 8 bits
per element (independently of the
value m) was sufficient to get $FPR = 0.02$

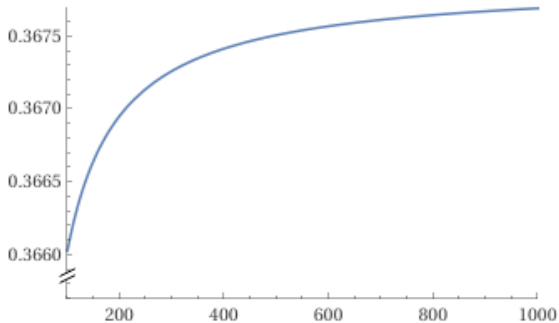


plot

$$\left(1 - \frac{1}{n}\right)^n$$

$n = 100$ to 1000

Plot



Hash functions

Hash functions are fundamental ingredients of many data processing tools: e.g., hash tables, sketches, filters, signatures, integrity checks.

A hash function h usually maps the elements of a universe U (e.g., all 64-bit numbers) into hash values represented by the integers in the range $[m] = \{0, 1, \dots, m-1\}$, for some $1 \leq m \leq |U|$. (usually $m \ll |U|$)

To simplify the analysis, it is often assumed that h is a truly random hash function, in the sense that $\{h(x) : x \in U\}$ is a set of $|U|$ independent random variables with uniform distribution over $[m]$

Hash functions

.

A truly random hash function can be regarded as a random variable with uniform distribution over $[m] \times [m] \times \cdots \times [m]$ ($|U|$ times).

$\Rightarrow$ representing a truly random hash function requires $\geq |U| \log_2 m$ bits

$$\log_2 m^{|U|} = |U| \log_2 m$$

In practice, we need hash functions which provide sufficient randomness for the application of interest, and are efficient in terms of

- **Space:** number of bits required by their representation
- **Speed:** time to compute the hash values.

k-universality

Definition: k-universality

A family $\mathcal{H}$ of hash functions from U to $[m]$ is said to be *k-universal* if for any k distinct elements $x_1, x_2, \dots, x_k$ from U , we have that for a hash function h extracted from $\mathcal{H}$ uniformly at random

$$\Pr(h(x_1) = h(x_2) = \dots = h(x_k)) \leq \frac{1}{m^{k-1}}.$$

Moreover, $\mathcal{H}$ is said to be *strongly k-universal* if for any k distinct elements $x_1, x_2, \dots, x_k$ of U , and any k values $y_1, y_2, \dots, y_k$ from $[m]$, we have that for a hash function h extracted from $\mathcal{H}$ uniformly at random

$$\Pr((h(x_1) = y_1) \wedge (h(x_2) = y_2) \wedge \dots \wedge (h(x_k) = y_k)) = \frac{1}{m^k}$$

Remark. Using hash functions randomly extracted from a 2-universal family yield the same theoretical guarantees as truly random functions for: hash tables with chaining, *count-min sketch*, signature.

Practical hash functions

Let the universe U be the set of integers $[u] = \{0, 1, \dots, u-1\}$, and $[m]$ be the range. Fix a **prime number** $p > u$. For $1 \leq a \leq p-1$ and $0 \leq b \leq p-1$ define the function $h_{a,b} : [u] \rightarrow [m]$ so that

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m \quad \text{for every } x \in [u].$$

Theorem

The family

$$\mathcal{H}_p = \{h_{a,b} : 1 \leq a \leq p-1, \text{ and } 0 \leq b \leq p-1\}$$

is 2-universal

bits needed to represent any $h_{a,b}$ is $\Theta(\log_2 p)$
and if p is not too large w.r.t. u , $\Theta(\log_2 u)$ bits
suffice

Practical hash functions

Convenient primes to be used for the family defined in the previous slide, are the so called **Mersenne primes**, that is, primes of the form $p = 2^q - 1$. For example,

$$p = 2^{13} - 1 = 8191$$

$$p = 2^{61} - 1$$

$$p = 2^{89} - 1$$

are all Mersenne primes. The latter two, can be used when U is the set of 32-bit integers or the set of 64-bit integers, respectively.

The **advantage of using a Mersenne prime** $p = 2^q - 1$ is that the modulo operation is simplified by the relation

$$x \bmod p = ((x \bmod 2^q) + \lfloor x/2^q \rfloor) \bmod p,$$

which reduces the modulo to fast bit shifts and additions, thus yielding an efficient implementation.

Exercise: convince yourself that the computation of $x \bmod p$ is fast

Exercise

Consider a stream $\Sigma = x_1, x_2, \dots, x_n$ of n measurements from sensors. Each measurement x_i is a pair (k_i, w_i) , where k_i is the ID of a sensor and w_i is the value of the measurement (an integer). For a given sensor u occurring in Σ define

$$f_u = \sum_{(k_i, w_i) \in \Sigma : k_i = u} w_i,$$

i.e., the aggregate measurements taken by u .

- 1 Briefly describe a space-efficient unbiased estimator for $\sum_u (f_u)^2$, where the sum is over all sensors occurring in the stream.
- 2 What can you say about the unbiasedness of your estimator?

Exercise

Let $\Sigma = x_1, x_2, \dots, x_n$ be a stream of $n > 10$ items (n even), which contains $(n/2) - 5$ occurrences of item a , $(n/2) - 5$ occurrences of item b , and 10 occurrences of other items $\neq a, b$. Let C be a **Count-min sketch** of size $d \times w$ for Σ . We define the following notation for item a :

- f_a = true frequency of a in Σ ;
- $\tilde{f}_{a,j}$ = the estimate of f_a provided by row j of C , with $0 \leq j < d$;
- $\tilde{f}_a$ = the final estimate of f_a provided by the sketch C .

Answer to the following points, **assuming** $w = 2$.

- ① Suppose that in row j , a and b are mapped to a different counter. Give a tight upper bound R to the relative error $(\tilde{f}_{a,j} - f_a)/f_a$.
- ② Inspired by the previous point, compute the following probabilities
 - $\Pr(\tilde{f}_{a,j} - f_a)/f_a \leq R$, for any given row j , with $0 \leq j < d$.
 - $\Pr(\tilde{f}_a - f_a)/f_a \leq R$.

Exercise

Consider a Bloom filter built to assess membership for a set S of m elements. The Bloom filter consists of a n -bit array A and k hash functions $h_0, h_1, \dots, h_{k-1}$, mutually independent and with values uniformly distributed in $[0, n-1]$. Assume that n is even, and that each hash function h_i is such that, for every $e \in S$, $h_i(e) \bmod n/2$ is uniformly distributed in $[0, n/2-1]$.

- 1 Show how to transform, in $O(n)$ time, the given Bloom filter into a new Bloom filter based on an $n/2$ -bit array B , and describe how to assess membership with the new Bloom filter.
- 2 Compute the probability that a given cell $B[i]$ of the new array is 0.

References

- **[LRU14]** J. Leskovec, A. Rajaraman and J. Ullman. **Mining Massive Datasets**. Cambridge University Press, 2014. Chapter 4 (Sections 4.3-4.5) (pdf provided in Moodle)
- **[DF08]** C. Demetrescu and I. Finocchi. **Algorithms for Data Streams**. In *Handbook of Applied Algorithms*, Chapter 8, Section 3. Wiley-IEEE Press, 2008. (pdf provided in Moodle)
- **[CGHJ12]** G. Cormode, M.N. Garofalakis, P.J. Haas, C. Jermaine. **Synopses for Massive Data: Samples, Histograms, Wavelets, Sketches**. Foundations and Trends in Databases 4(1-3): 1-294, 2012. Chapter 5 (Sections 5.1-5.3) (pdf provided in Moodle)